Interfaces or Bodies? A Preregistered Test of ε-Sufficient Context for Language-Model Code Editing in Go

Jude Castiel Johansen
Shockware Limited, independent researcher
castiel@shockware.co.uk ORCID 0009-0002-5644-3073

***Abstract*—Coding agents decide, many times a day, how much of a repository to show a language model, and the decision is made by heuristic or by end-task scores that confound localisation, context size, and model quality. A recent note defines *ε-sufficient context*: a context is sufficient for an editor when the editor's success on it is within ε of its success on the best context in a stated family, and proves that under a bounded reader this property is not closed under superset. We propose a preregistered, paired, multi-model study that operationalises the definition as a non-inferiority test. Holding localisation fixed at the gold write-set on at least 200 fail-to-pass instances mined from a fixed, ranked list of Go repositories after each model's training cutoff, we compare (H2) a compiler-computed context consisting of the write-set and the exported interfaces of its dependencies against the same package set with full bodies and against the transitive slice; (H1) we estimate each model's degradation curve by padding a fully covered base context to fixed absolute loads up to 950k tokens with three padding types, with a pre-specified equivalence test so that a flat curve is a reportable result; (H3) we measure, descriptively, whether showing the forward slice reduces breakage of passing tests; and (H4) we report tokens per arm. Coverage is defined operationally per arm by checking that the gold patch is expressible from the context. Three models: a dense cheap model at ten samples per cell on every arm, an open-weights model at temperature zero, and a frontier model at vendor-default sampling on a fixed subset. A 24-instance pilot on one repository (2,360 verified samples, three models, about $61, published with all data) shaped this design: it fixed the margins from measured per-instance variance, made a task-only screen mandatory after the mid-tier model solved 14 of 24 instances with no repository context, extended the load range after padding to 200k produced no consistent effect, and set the sampling depth after a quarter of the dense model's cells disagreed across samples. The instances, slices, contexts and verdicts are released as SuffBench v1.0.**

I. Introduction

Every system that uses a language model to edit code must choose which parts of the repository to place in the model's context. The choice is consequential in two directions. Too little, and the model lacks a fact it needs; too much, and its accuracy falls with input length even when the needed facts are present [1], [2], [3]. Current practice chooses by retrieval heuristics or by the agent's own exploration [4], [5], and evaluates the choice only through the end-task pass rate, which confounds whether the right place was found, how much was shown, and how good the model is.

A recent definitions note [6] separates these. A change request is a contract (Δ, D, I, W): requested behaviour, scope of permitted behavioural change, invariants, and a write-set. A context K is *ε-sufficient* for an editor A when A's success on K, judged by an oracle of tests, is within ε of its success on the best context in a stated candidate family. Under a stylised model in which success factors into coverage of needed facts and a non-increasing function of load, the note proves, with a Lean 4 mechanisation, that sufficiency is not closed under superset: adding irrelevant material to a sufficient context can make it insufficient, and the admissible tolerance grows with the amount added. The note also states three properties a language could enforce so that the sufficient context becomes a graph query: dependencies matter only through their declared interfaces (P1); dependents rely only on what interfaces pin down (P2); and the context is computable from the dependency graph before the edit (P3). A mutation study on one Go repository found that interface-preserving faults were caught by tests outside the changed package in about one case in twenty-three, concentrated in shared helper packages.

None of that is evidence about real editors. A 24-instance pilot [7], described in Section V, gives first estimates and shaped this design; it is exploratory and is not reused. This report proposes the study that would supply confirmatory evidence, designed to be cheap enough for an independent researcher and powered enough to be worth running. Its central question is P1 as an experiment: does an editor given only the interfaces of what the write-set depends on do as well as one given the bodies, and as well as one given the transitive slice? Framed as non-inferiority, this is exactly the ε-sufficiency definition. Around it we estimate the degradation curve per model at verified fixed coverage, measure the forward slice's effect on regressions descriptively, and report the token cost of every arm.

A. Relation to recent work

Several 2026 measurements bear directly on the design and we build on them rather than claim to precede them. Sam-Bodden [8] holds localisation fixed on SWE-bench Verified and varies the representation of surrounding Python code, finding that signatures and skeletons serve about as well as source; his preregistered superiority test failed to find a difference, which is precisely the gap a powered non-inferiority test fills. Mohammadi et al. [9] show across seven models that availability of needed facts decides success and their position

does not. Du et al. [3] show that length alone hurts even under perfect retrieval and neutral padding. Xue [10] reports large drops from short to long contexts on two auditing tasks. Alonso et al. [11] reduce agent regressions on SWE-bench Verified from about six percent to under two with a graph-based impact map. Jia, Barr and Mechtaev [12] search for minimal sufficient context on the editor side. Hierarchical context pruning [13] found that pruning function bodies in dependent files did not significantly reduce completion accuracy. Bjarnason et al. [14] quantify run-to-run randomness in agent evaluation, which bounds what any paired comparison can resolve and motivates our pilot.

What remains open, and what this study measures: the interfaces-versus-bodies question as a powered non-inferiority test in a statically typed language with type-graph-derived interfaces; a per-model degradation curve on repository editing with verified coverage and intervals; the forward slice's effect in a single-shot setting stratified by whether the write-set is a shared helper; and a dataset whose relevance labels come from test verdicts rather than from workflow signals or gold-patch files [15], [16].

II. Hypotheses and estimands

All contrasts are paired by instance. p_X denotes the population-average probability, over instances and samples, that a patch produced from context X passes the fail-to-pass tests (Δ-pass). Where stated, the outcome is instead the conjunction Δ-pass and frame-pass (all pass-to-pass tests still pass).

- **H2 (primary): interface sufficiency.** $p_{\mathrm{A1}} \geq p_{\mathrm{A1b}} - \varepsilon$ and $p_{\mathrm{A1}} \geq p_{\mathrm{A2}} - \varepsilon$, where A1 is write-set bodies plus interfaces of direct dependencies, A1b the same package set with bodies, and A2 the transitive slice. Margins are fixed now (Section V): $\varepsilon = 0.05$ for the dense model and $\varepsilon = 0.075$ for the open-weights model; these two models carry the confirmatory test. The frontier model, run on a subset, replicates H2 descriptively with intervals. Estimand: population-average over the confirmatory instance set, defined in Section III. Per-instance ε-sufficiency is estimated on the dense model only ($k = 10$).
- **H1: degradation.** At fixed, verified coverage, Δ-pass falls with absolute load. Two pre-specified tests per load level $L \in \{200\mathrm{k}, 400\mathrm{k}, 950\mathrm{k}\}$ and padding type: a one-sided paired superiority test of A1 over A1 padded to L (degradation), and a two one-sided tests (TOST) equivalence test at $\pm\delta$, $\delta = 0.05$ (flatness). Each level has three possible outcomes, degradation, flat, or inconclusive, all reportable. The full curve at {base, 64k, 200k, 400k, 700k, 950k} is reported with intervals. Levels above a model's context window are not built for that model, and the instance set for each (model, level) is fixed before any call as the instances for which that arm can be built.
- **H3 (descriptive): forward slice.** The rate of Δ-pass-and-frame-pass for A1F (A1 plus interfaces and test names of the forward slice) versus A1, overall and stratified by whether the write-set is a shared helper (number of dependents at or above the median over the confirmatory set). Reported with intervals; descriptive regardless of the number of frame failures.
- **H4: cost.** Tokens, latency, and price per arm; the ratio of A1 to A2 and to lexical retrieval at matched and at large budgets.
- **Control: parametric memory.** $p_{\mathrm{A}\emptyset}$, task text only. In the pilot the mid-tier model solved 14 of 24 instances this way (11 of those 24 task texts were model-rewritten specifications, 13 were issue bodies). The task-only arm is therefore a screen, run before any other arm on the frontier model (the strongest evaluated) with $k = 3$: any instance it solves at least once is excluded from the confirmatory set and reported separately. The estimand of every hypothesis is accordingly the population of instances that the frontier model cannot solve from the task text alone.

III. Instances: SuffBench v0.1

Sources. A ranked list of single-module Go repositories is fixed here and consumed in order until the instance target is met: cli/cli, gohugoio/hugo, docker/cli, golangci/golangci-lint, restic/restic, rclone/rclone, syncthing/syncthing, jesse-duffield/lazygit, go-gitea/gitea, prometheus/prometheus. Before mining, each must build and pass its full suite on the Linux reference container; a repository that fails is skipped with the reason logged and the next taken (the pilot dropped Caddy because its tests install a root certificate and zap because core packages fail at baseline). Packages that fail at baseline on the reference container are excluded from the frame check and listed. The Multi-SWE-bench harness and container recipes [17] are reused. Pilot instances from cli/cli are excluded.

Mining. For each first-parent commit after the latest documented training cutoff of any evaluated model (cutoff dates recorded in the preregistration at the time the models are fixed), diffed against its first parent and touching both non-test and test files, with no `go.mod` or vendor change, no deleted or renamed source file, at most two changed non-test packages, and at most 120 changed non-test lines: the child's test files are applied to the parent; the changed test packages must compile there and at least one test must fail; at the child those packages, the write-set and every package depending on it must be green. Instances whose added test lines reference tokens (identifiers or string literals) that the gold patch introduces into the write-set are excluded. Signatures are never supplied as specification in this study. The write-set is the set of packages whose non-test files the gold patch changes; the interface write-set is computed by exported-API diff.

Task text. The linked issue body where one exists; otherwise a behaviour-only specification rewritten from the pull-request text by a model that is not among the evaluated models, with all identifiers and file names removed, hand-checked by the author. The source is recorded and results are reported by source.

Screening and the confirmatory set. Every mined instance is screened manually for under-specification and leakage, about ten minutes each, following SWE-bench Verified's protocol; the pilot's automatic leak rule rejected none of 25 candidates while manual inspection vetoed one, so both are required. Then the task-only screen above is run. The confirmatory set is the screened instances for which A1, A1b and the expressibility check succeed, fixed and hashed before any other model call; instances for which A2 or a padding level cannot be built remain in the set and are simply absent from that arm. Target: at least 200 confirmatory instances (stretch 300), with two-package write-sets at least 25%. From the pilot's yield (one valid instance per twelve commits) and screen loss (14 of 24 on the mid-tier model), 200 instances need about 500 valid candidates from about 6,000 commits and about 80 hours of manual screening. If fewer than 200 survive after the tenth repository, the study runs on what survived and the shortfall is reported with the power it implies.

Slices and checks. For each instance: backward and forward slices from `go list -deps -test`; a coverage slice from per-package coverage of the verdict tests on the gold tree; a mutation slice using body-only, interface-preserving operators; a closure check that the write-set with its backward and forward slices covers the import closure of the executed test binaries; and, per arm, an *expressibility* check that every identifier, declaration, and import the gold patch references is present in the context. Instances where the gold patch adds imports outside the parent's dependency graph are flagged.

Flakiness. Every test that fails on any run is rerun three times on the clean tree; those failing on the clean tree are excluded and listed.

Release. Under embargo until Stage 1 submission; then on Zenodo with the datasheet required by the public SuffBench specification [18].

IV. Design

Arms. Table I. Localisation is fixed at the gold write-set W in all arms but the retrieval baselines. Contexts are serialised deterministically with W last, immediately before the task. Padding packages are same-module packages outside W, its backward slice and its forward slice, not sharing a parent directory with any W package, taken in an order fixed by the SHA-1 of the instance identifier and import path, placed before A1 and cut at a line boundary to within 2% of the target as measured by the vendor's token counter for the dense model; cross-repository padding takes Go source from a fixed, listed repository not in the mining list; prose padding takes paragraphs of Go release notes in a fixed order.

Editor protocol. Single shot: task text and context in, edits out in a search-and-replace block format, applied with a strict matcher. One format-repair turn is allowed only when a block fails to apply; no test feedback is ever given. Open-weights models run at temperature zero; the frontier models' sampling parameters (temperature, top-p, top-k) are not settable, so each of their samples is an independent call at vendor defaults and run-to-run variation is estimated from the samples. Samples per cell: ten on the dense model for every arm; three on the open-weights and frontier models for A$\emptyset$, A1, A1b and A2, one for every padded and retrieval arm. Identical prompts share a prompt cache, which is what made the pilot's 2,360 samples cost $61. Provider and quantisation are pinned and recorded.

Arm	Content	Median prompt toke
A$\emptyset$	task text only	0.
A0	W bodies	1
A1	W bodies + interfaces of deps(W)	4
A1b	W bodies + bodies of deps(W)	12
A1F	A1 + interfaces and test names of $F(W)$	5
A2	W + bodies and tests of $B(W) \cup F(W)$	not run in pi
A4s / A4l	lexical retrieval at A1's / A2's budget	not run in pi
P_{64}..P_{950}	A1 padded to 64k, 200k, 400k, 700k, 950k	as nam

TABLE I

Arms. Median prompt tokens are from the pilot (24 cli/cli instances, dense-model tokenizer, system prompt and task included). A2 in the pilot's two validation instances was 0.4M and 2.9M tokens; arms above a model's window are not built for it.

Models. Fixed at the time of in-principle acceptance and recorded with documented training cutoffs in the preregistration: a dense cheap model (Claude Haiku 5.5 or its successor) on all arms within its window and all instances; an open-weights model with a one-million-token window via a hosted API with pinned provider and quantisation, on all arms and all instances; a frontier model (Claude Opus 5 class) on A$\emptyset$ (all instances, as the screen), A1, A1b, A2 within its window and P_{200} over a stratified random 100-instance subset (strata: repository and write-set size; seed 20261120), and P_{950} over a 30-instance sub-subset. If a fixed model is withdrawn before the run, its nearest successor from the same vendor is substituted and the substitution is dated in the preregistration.

Verification. Stage one runs the fail-to-pass tests; stage two runs the full suite only for patches that pass stage one. Outcomes: applied, Δ-pass, frame-pass, tokens in and out, latency.

V. Analysis plan

Pilot. Twenty-four instances mined from 282 GitHub CLI commits by the pilot's own rules (the mining rules above, except that the baseline was the author's Windows machine with 13 failing packages excluded, cutoffs were not verified, and the frame check was scoped to dependents of the write-set), three models, 2,360 verified samples, published with all data [7]. Findings that bind this design: (i) on Haiku 5.5 over 24 instances, A1 had a higher Δ-pass rate than A1b by $+0.042$ [$+0.012$, $+0.075$] and than W by $+0.067$ [$+0.021$, $+0.133$], with no majority-outcome disagreement between A1 and A1b; the pilot paper notes that A1 was identified as best after the fact and the contrasts are uncorrected; (ii) padding A1 to 200k changed Δ-pass by -0.03 [-0.15, $+0.09$] on Haiku and by $+0.04$ [-0.22, $+0.33$] on Sonnet 5.5 and $+0.07$ [0.00, $+0.19$] on Opus 5, the latter two on nine instances selected as those Sonnet had not solved from task text; (iii) Sonnet 5.5 solved 14 of 24 instances from the task text alone; (iv) 25% of Haiku cells ($k = 10$), 11% of Sonnet cells and 7%

of Opus cells ($k = 3$) were non-unanimous, with the minority outcome holding about 30% of samples; (v) nine instances were ceilings on which the write-set alone always passed. Pilot data are exploratory and are not reused.

Margins, fixed now. $\varepsilon = 0.05$ for the dense model and $\varepsilon = 0.075$ for the open-weights model. Justification from the pilot's per-instance paired differences: on Haiku the sample standard deviation of A1−A1b over 24 instances is 0.083, so a one-sided paired test at $\alpha = 0.05$ has 80% power at a true difference of zero for $n \geq 6.18\,\sigma^2/\varepsilon^2 \approx 17$; at $n = 200$ power exceeds 0.99 even at $\sigma = 0.35$. For models at $k = 3$ the pilot's only paired contrast is the padding one: the sample standard deviation of P200−A1 is 0.26 over the 22 Sonnet instances with both arms and 0.42 over the nine that survived the task-only screen; at $\varepsilon = 0.075$ and $n = 200$ power is 0.99 at $\sigma = 0.26$ and 0.81 at $\sigma = 0.42$. The frontier subset ($n = 100$) has power 0.56 at $\sigma = 0.42$, which is why its H2 is descriptive. These values do not change after the first confirmatory model call.

Tests. The unit is the instance; a cell's value is its mean Δ-pass over samples. The deciding interval everywhere is the instance-level percentile bootstrap of the mean per-instance difference (2,000 resamples, seed 1, 95% two-sided, equivalently 97.5% one-sided bounds); a Tango score interval on majority outcomes is reported alongside. H2 is an intersection–union test: both comparisons must have a lower bound above $-\varepsilon$; being intersection–union it is level $\alpha = 0.05$ without further adjustment and is tested first and alone. H1 forms one family per model of nine superiority tests (three levels $\times$ three padding types) under Holm at $\alpha = 0.05$; the TOST equivalence tests at ± 0.05 are reported per level without correction, as the pre-specified reading of a null superiority result. Intervals on single proportions are Wilson intervals computed on per-instance means, so samples within an instance are not treated as independent. Pass rates across padding levels are compared only through paired differences, since levels are built for different instance subsets.

No interim stopping. The sample size is fixed in advance.

Reporting. All arms, all models, all instances, in a public table; every pre-specified contrast whether or not significant; every deviation from this plan, with reasons, in a dedicated section.

VI. Threats and limitations

Single-shot editing is not an agent loop; the study measures context, not agency, and says so. Three models and one language. Gold localisation is a ceiling; the retrieval arms are the realistic floor. Task text may leak the fix despite screening; results are reported by source and the parametric-memory arm bounds the effect. Padding from the same repository can raise coverage; the similarity ceiling and the two other padding types address this. Temperature zero is not deterministic across providers [14], and the frontier models offer no such control; the pilot measured between 7% and 25% non-unanimous cells at default sampling depending on model and k, and the margins above are built on per-instance variance rather than on that rate. The pilot's write-sets were mostly single packages with a median of seven changed lines and nine of 24 were ceilings, so its variance estimates may understate the screened population's; power is therefore reported at σ up to 0.42. The pilot's own task texts were partly written by an evaluated model; here the rewriting model is excluded from evaluation. Padding above 200k may be priced at a higher tier; the budget below carries that as its upper bound. Coverage is checked syntactically through expressibility; semantic needs beyond identifiers are not captured. The forward-slice question is likely under-powered and is labelled descriptive.

VII. Timeline and resources

Mining, screening, and slices: weeks 1 to 8 (about 80 hours of manual screening at the pilot's yield of one valid instance per twelve commits and a 58% task-only screen loss). Harness: complete and released with the pilot. Stage 1 submission: 20 November 2026. Confirmatory run after in-principle acceptance: three weeks. Analysis and Stage 2: by the EMSE deadline. Budget, from the pilot's measured unit costs ($0.20 per instance for nine arms at ten samples on the dense model; $1.22 per instance for five arms at three samples on a mid-tier model; prompt caching makes samples after the first nearly free) and from list prices for the arms the pilot did not run: dense model, all arms within its window, 200 instances, about $150; open-weights model at hosted list prices without caching, about $500; frontier model on the 100-instance subset plus the 30-instance P_{950} sub-subset, about $900. About $1,600 in total, with an upper bound of $2,200 if input above 200k tokens is billed at a higher tier; verification on a laptop. All code, data, and prompts are released under Apache-2.0 and CC-BY-4.0.

AI-use disclosure

Anthropic's Claude (Fable 5.1, referred to by the author as Lemma) was used under the author's direction to draft this report, to write the analysis, mining and harness scripts, to run the pilot against the author's API account, and to cross-check references; separate Claude Opus agents produced several rounds of critical review of the design and of the pilot paper, whose findings the author adopted or rejected. The author reviewed all text and takes full responsibility for it. Generative AI tools are not authors.

References

[1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[2] M. Levy, A. Jacoby, and Y. Goldberg, "Same task, more tokens: The impact of input length on the reasoning performance of large language models," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL 2024)*, 2024, pp. 15 339–15 353.

[3] Y. Du, M. Tian, S. Ronanki, S. Rongali *et al.*, "Context length alone hurts LLM performance despite perfect retrieval," arXiv:2510.05381, 2025.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 801–824, 2025, describes the Agentless approach.

[5] R. Bairi, A. Sonwane, A. Kanade, V. D. C., A. Iyer, S. Parthasarathy, S. Rajamani, B. Ashok, and S. Shet, "CodePlan: Repository-level coding using LLMs and planning," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 675–698, 2024.
[6] J. C. Johansen, "ε-sufficient context for bounded readers: Definitions and a non-monotonicity result for code editing," Zenodo, version 1.1, 2026.
[7] ——, "Interfaces or bodies? a 24-instance pilot measurement of ε-sufficient context for language-model code editing in Go," Zenodo, Oct. 2026, pilot report v1.0; data and harness at https://github.com/Shockware-Limited/suffbench, tag v0.1.1-pilot.
[8] B. Sam-Bodden, "What context does a coding agent actually need to act?" arXiv:2607.09691, 2026.
[9] B. Mohammadi, L. Klein, A. Chadha, A. Arora *et al.*, "The working set of a coding agent: Coherence debt in repository-scale tasks," arXiv:2608.16630, 2026.
[10] Y. Xue, "When and how context rot appears in coding agents: A white-box study of agent skills in code auditing," arXiv:2607.17937, 2026.
[11] P. Alonso, S. Yovine, and V. A. Braberman, "TDAD: Test-driven agentic development: Reducing code regressions in AI coding agents via graph-based impact analysis," arXiv:2603.17973, 2026.
[12] H. Jia, E. T. Barr, and S. Mechtaev, "Compressing code context for LLM-based issue resolution," arXiv:2603.28119, 2026.
[13] L. Zhang, Y. Li, J. Li, X. Xia *et al.*, "Hierarchical context pruning: Optimizing real-world code completion with repository-level pretrained code LLMs," arXiv:2406.18294, 2024.
[14] B. H. Bjarnason, A. Silva, and M. Monperrus, "On randomness in agentic evals," arXiv:2602.07150, 2026.
[15] B. Qin and Y. Xie, "Agent retrieval bench: Evaluating repository context retrieval for coding agents," arXiv:2607.24882, 2026.
[16] H. Li, L. Zhu, B. Zhang, R. Feng *et al.*, "ContextBench: A benchmark for context retrieval in coding agents," arXiv:2602.05892, 2026.
[17] D. Zan, Z. Huang, W. Liu, H. Chen *et al.*, "Multi-SWE-bench: A multilingual benchmark for issue resolving," arXiv:2504.02605, 2025.
[18] J. C. Johansen, "SuffBench: Specification for a context-sufficiency benchmark for code editing (v0.1.0-spec)," Shockware Limited, https://github.com/Shockware-Limited/suffbench, 2026.